

Programare Orientată Pe Obiecte

Curs 10

Dezvoltarea Aplicațiilor cu Interfață Grafică în Java
(GUI)

Decembrie 2024

Introducere

GUI

O interfață utilizator grafică - Graphical User Interface (GUI) prezintă un mecanism prietenos pentru interacțiunea utilizatorului cu un program

- GUI dă programului un aspect ("look") și un mod în care este "simțit" ("feel") caracteristic
- Permite utilizatorilor să se simtă mai familiarizați cu programul chiar înainte de a-l fi utilizat
- Reduce timpul de învățare a modului de utilizare

Java oferă mai multe pachete pentru dezvoltarea de aplicații GUI. Cele mai importante pachete GUI sunt:

- 1 AWT (Abstract Window Toolkit)
- 2 Swing
- 3 JavaFX

AWT (Abstract Window Toolkit)

- Pachetul original pentru GUI în Java.
- Componente de bază precum butoane, etichete, câmpuri text etc.
- Exemple de clase:
 - `Frame`, `Panel`, `Button`, `TextField`, `Label`.
- Gestionarea evenimentelor: `ActionEvent`, `MouseEvent`.
- **Utilizare:** Aplicații GUI simple și ușoare.

Swing

- Toolkit avansat, construit peste AWT.
- Oferă mai multe componente și control sporit asupra designului.
- Exemple de clase:
 - JFrame, JPanel, JButton, JTable, JTree.
- Suportă:
 - Pluggable Look and Feel (ex.: Metal, Nimbus).
 - Arhitectura MVC pentru componente precum JTable.
- **Utilizare:** Majoritatea aplicațiilor desktop în Java.

JavaFX

- Toolkit modern, succesor al Swing.
- Caracteristici avansate:
 - Grafică 3D, animații.
 - CSS pentru stilizare.
- Exemple de clase:
 - Stage, Scene, Button, TableView.
- Suportă design declarativ cu FXML.
- **Utilizare:** Aplicații moderne cu interfețe bogate și vizualizări avansate.

Comparație între pachete GUI

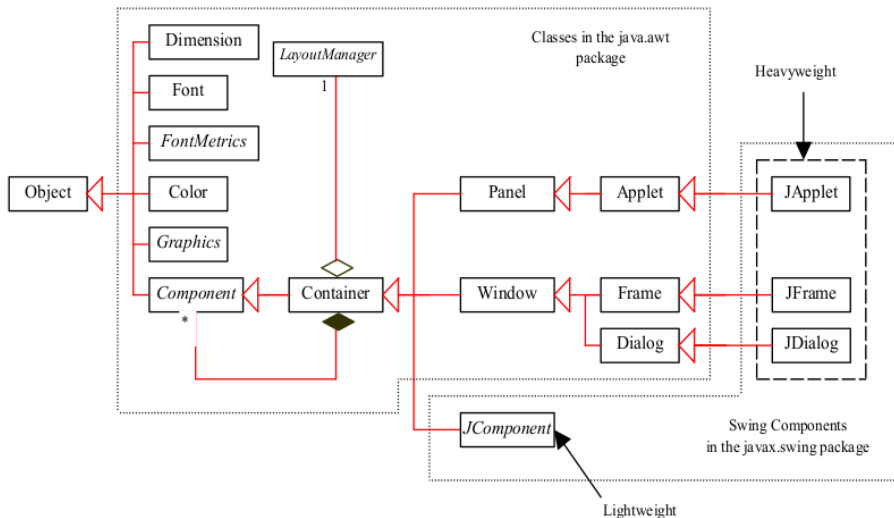
Caracteristică	AWT	Swing	JavaFX
Vechime	Cel mai vechi	Mai vechi	Modern
Aspect	Simplu	Personalizabil	Modern
Ușurință în utilizare	Simplu	Moderat	Avansat
Performanță	De bază	Moderată	Ridicăată

Recomandare:

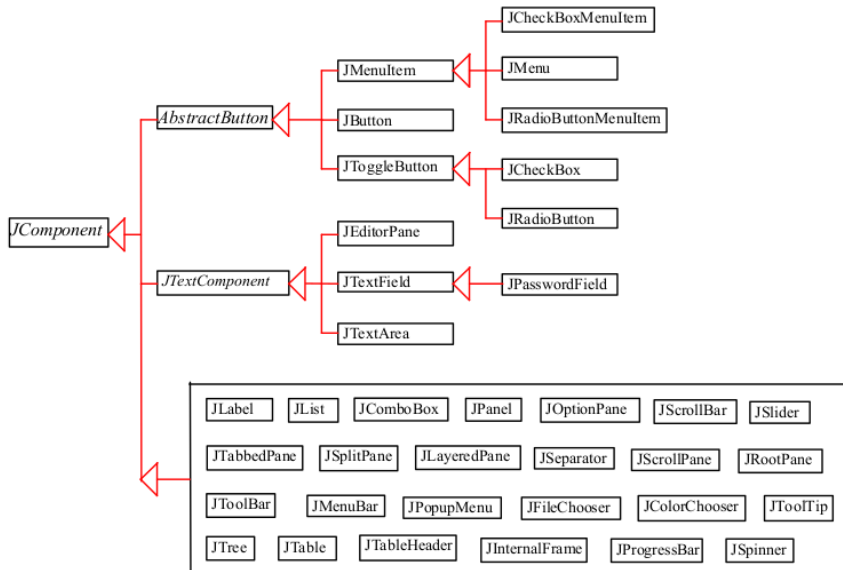
- Pentru aplicații moderne, se recomandă utilizarea **JavaFX**.
- **Swing** este încă utilizat pe scară largă, dar mai puțin potrivit pentru aplicații noi.

Swing

Ierarhia de clase Swing



Componentele Swing



Exemplu 1: Fereastră simplă

```
1 import javax.swing.*;
2 public class Main {
3     public static void main(String[] args) {
4         JFrame frame = new JFrame("Hello Swing");
5         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         frame.setSize(200,200);
7         frame.setVisible(true);
8     }
9 }
```

Exemplu 1: Fereastră cu componente

```
1 import javax.swing.*;
2 public class Main {
3     public static void main(String[] args) {
4         JFrame frame = new JFrame("Hello Swing");
5         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         frame.setSize(700,150);
7         JPanel panel = new JPanel();
8         JButton jbtOK = new JButton("OK");
9         panel.add(jbtOK);
10        JLabel jlblName = new JLabel("Enter your name: ");
11        panel.add(jlblName);
12        JTextField jtfName = new JTextField("Type Name Here");
13        panel.add(jtfName);
14        JCheckBox jchkBold = new JCheckBox("Bold");
15        panel.add(jchkBold);
16        JRadioButton jrbRed = new JRadioButton("Red");
17        panel.add(jrbRed);
18        JComboBox jcboColor = new JComboBox(new String[]{"Red", "Green", "
19        Blue"});
20        panel.add(jcboColor);
21        frame.setContentPane(panel);
22        frame.setVisible(true);
23    }
```

Exemplu 2: Fereastră cu componente

Moștenește JFrame, C10-gui-2

```
1 import javax.swing.*;
2 public class MainWindow extends JFrame {
3     JButton jbtOK = new JButton("OK");
4     JLabel jlblName = new JLabel("Enter your name: ");
5     JTextField jtfName = new JTextField("Type Name Here");
6     JCheckBox jchkBold = new JCheckBox("Bold");
7     JRadioButton jrbRed = new JRadioButton("Red");
8     JComboBox jcboColor = new JComboBox(new String[]{"Red", "Green", "
        Blue"});
9     MainWindow(){
10         this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         this.setSize(700,150);
12         JPanel panel = new JPanel();
13         panel.add(jbtOK);
14         panel.add(jlblName);
15         panel.add(jtfName);
16         panel.add(jchkBold);
17         panel.add(jrbRed);
18         panel.add(jcboColor);
19         this.setContentPane(panel);
20     }
21 }
```

Exemplu 2: Fereastră cu componente

Moștenește JFrame, C10-gui-2

```
1 public class Main {  
2     public static void main(String[] args) {  
3         MainWindow mw = new MainWindow();  
4         mw.setTitle("Exemplu GUI 2");  
5         mw.setVisible(true);  
6     }  
7 }
```

Containere și componente

- Clasa **Container** gestionează o colecție de componente înrudite
 - în aplicații care folosesc JFrame și în applet-uri atașăm componente panoului de conținut (content pane) – care este un container
 - metode importante: add(), setLayout()
- Clasa **Component** declară atributele și comportamentele comune tuturor subclaselor sale
 - Metode importante: paint(), repaint()

Clasa **Container** și Clasa **JComponent**

Container

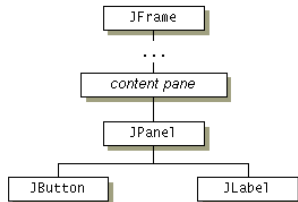
- Orice clasă care descinde în clasa Container este considerată o clasă container
- Clasa Container se află în pachetul java.awt, nu în Swing
- Oricărui obiect care aparține unei clase derivate din clasa Container (sau din descendenții săi) i se pot adăuga componente
- Clasele JFrame și JPanel sunt descendente din clasa Container
- De aceea ele și orice alți descendenți ai lor pot servi pe post de container

JComponent

- Orice descendent al clasei JComponent se numește clasă componentă
- Oricare obiect JComponent sau component poate fi adăugat la orice obiect de clasă container
- Deoarece este derivată din clasa Container, o JComponent poate fi

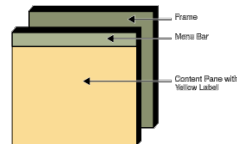
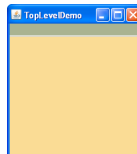
Ierarhii de containere

- Containere de nivel înalt
 - Containere intermediare
 - Componente atomice
- Containere de nivel înalt:
 - Se află la rădăcina fiecărei ierarhii de conținere
 - Toate programele Swing au cel puțin un container de nivel înalt
 - Panouri de conținut
 - Tipuri de containere de nivel înalt
 - Ferestrele (frames)
 - Dialoguri
 - Applet-uri

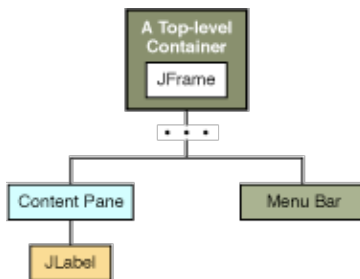


Panouri de conținut

- Folosesc de obicei un JPanel
- Conține totul cu excepția barei de meniu pentru majoritatea aplicațiilor Swing
- Poate fi creat explicit sau implicit



Corespunde la următoarea ierarhie de componente (codul JAVA e pe slide-ul următor):



Exemplu 3: Fereastra cu JMenuBar verde și etichetă galbenă

C10-gui-3

```
1 import javax.swing.*;
2 public class Main {
3     public static void main(String[] args) {
4         JFrame frame = new JFrame("Hello Swing");
5         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         frame.setSize(700,150);
7         JMenuBar greenMenuBar = new JMenuBar();
8         greenMenuBar.setOpaque(true);
9         greenMenuBar.setBackground(new Color(0, 200, 0));
10        greenMenuBar.setPreferredSize(new Dimension(200, 20));
11        //Creaza un pane
12        JPanel contentPane = new JPanel();
13        JLabel yellowLabel = new JLabel();
14        yellowLabel.setOpaque(true);
15        yellowLabel.setBackground(new Color(250, 250, 0));
16        yellowLabel.setPreferredSize(new Dimension(200, 180));
17        contentPane.add(yellowLabel);
18        frame.setJMenuBar(greenMenuBar);
19        frame.setContentPane(contentPane);
20        frame.setVisible(true);
21    }
```

Obiecte dintr-un GUI tipic

Aproape fiecare GUI construit folosind clasele container din Swing vor fi compuse din până la trei feluri de obiecte:

- 1 Containerul însuși, probabil un obiect panou (panel) sau de tip fereastră (window-like)
- 2 Componentele adăugate containerului, cum sunt etichetele (label), butoanele și panourile
- 3 Un gestionar de aranjare (layout manager) pentru a poziționa componentele în interiorul containerului

Gestiunea Aranjării

Gestiunea aranjării

În exemplele anterioare am folosit un control limitat asupra aranjării (layout) componentelor:

- Când am folosit un panou, acesta a aranjat implicit componentele de la stânga la dreapta

Componentele din interfața utilizator sunt aranjate prin plasarea lor în containere:

- Fiecare container are un gestionar de aranjare (layout manager) care dirijează aranjarea componentelor sale
- Câteva gestionare de aranjare utile: BorderLayout, FlowLayout, GridLayout, BoxLayout
- Gestionarul implicit este flowLayout
- Se pot seta alte gestionare de aranjare

```
1 panel.setLayout(new BorderLayout())
```

Gestiunea Aranjării

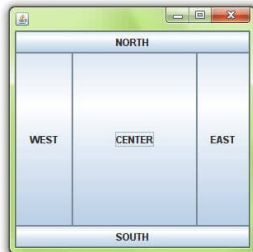
- Pasul 1: Facem o schiță a modului de aranjare dorit
- Pasul 2: Determinăm grupări de componente adiacente cu același mod de aranjare (layout)
- Pasul 3: Identificăm modul de aranjare pentru fiecare grup
- Pasul 4: Grupăm împreună grupurile
- Pasul 5: Scriem codul pentru generarea aranjamentului

Border Layout

- Aranjarea după margini (border layout) grupează în cinci zone: centru, nord, vest, sud și est
- Este **gestionarul de aranjare implicit pentru ferestre** (tehnic, pentru panoul de conținut al ferestrei)
- La adăugarea unei componente se specifică poziția astfel:

```
1 panel.add(component, BorderLayout.NORTH);
```

- Extinde fiecare componentă pentru a umple toată zona alocată
- Dacă nu doriți aceasta, atunci puneți fiecare componentă într-un panou



Exemplu: C10-gui-4

```
1 import javax.swing.*;
2 import java.awt.*;
3 public class Main {
4     public static void main(String[] args) {
5         JFrame frame = new JFrame("Layout Manager Demo");
6         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
7         frame.setSize(600, 600);
8         frame.setLayout(new BorderLayout());
9
10        JPanel flowLayoutP = new JPanel(new FlowLayout());
11        flowLayoutP.setBackground(new Color(240, 100, 145));
12        flowLayoutP.setBorder(BorderFactory.createTitledBorder("FlowLayout
13            "));
14
15        JPanel borderLayoutP = new JPanel(new BorderLayout());
16        borderLayoutP.setBackground(new Color(100, 200, 190));
17        borderLayoutP.setBorder(BorderFactory.createTitledBorder("
18            BorderLayout"));
19
20        JPanel gridLayoutP = new JPanel(new GridLayout(2, 3, 5, 5));
21        gridLayoutP.setBackground(new Color(100, 80, 210));
22        gridLayoutP.setBorder(BorderFactory.createTitledBorder("GridLayout
23            "));
```

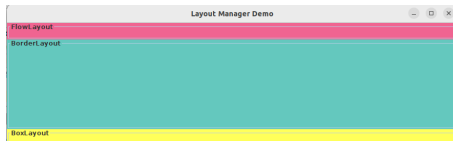
Exemplu: C10-gui-4

BorderLayout

```

1 // ... continuare main
2 JPanel boxLayoutP = new JPanel();
3 boxLayoutP.setBackground(new Color(255, 255, 90));
4 boxLayoutP.setBorder(BorderFactory.createTitledBorder("BoxLayout")
5     );
6
7 // Add all layout panels to the main frame
8 frame.add(flowLayoutP, BorderLayout.NORTH);
9 frame.add(borderLayoutP, BorderLayout.CENTER);
10 frame.add(gridLayoutP, BorderLayout.EAST);
11 frame.add(boxLayoutP, BorderLayout.SOUTH);
12 // Set the frame visible
13 frame.setVisible(true);
14 } }

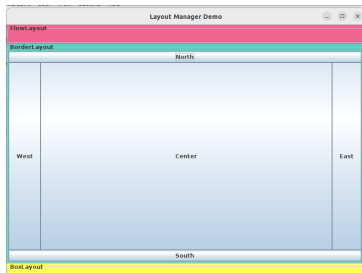
```



Exemplu: C10-gui-4

BorderLayout

```
1 // ... modificati in exemplul anterior:
2 JPanel borderLayoutP = new JPanel(new BorderLayout());
3 borderLayoutP.setBackground(new Color(100, 200, 190));
4 borderLayoutP.setBorder(BorderFactory.createTitledBorder("BorderLayout
   "));
5 borderLayoutP.add(new JButton("North"), BorderLayout.NORTH);
6 borderLayoutP.add(new JButton("South"), BorderLayout.SOUTH);
7 borderLayoutP.add(new JButton("East"), BorderLayout.EAST);
8 borderLayoutP.add(new JButton("West"), BorderLayout.WEST);
9 borderLayoutP.add(new JButton("Center"), BorderLayout.CENTER);
```



Gestionarul de aranjare FlowLayout

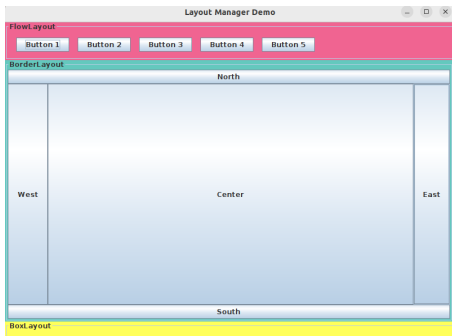
- Aranjează componentele în ordine de la stânga la dreapta și de sus în jos în container
- Constructori:

```
1 public FlowLayout();  
2 public FlowLayout(int align);  
3 public FlowLayout(int align, int horizontalGap, int verticalGap);
```

- Parametrul *align* poate fi LEFT, RIGHT, sau CENTER
- Acest layout este **implicit pentru JPanel**

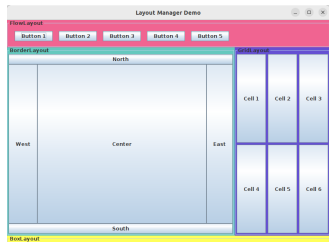
Gestionarul de aranjare FlowLayout: Exemplu

```
1 //... modificati in exemplul C10-gui-4
2 JPanel flowLayoutP = new JPanel(new FlowLayout(FlowLayout.LEFT,15,10))
3 flowLayoutP.setBackground(new Color(240, 100, 145));
4 flowLayoutP.setBorder(BorderFactory.createTitledBorder("FlowLayout"));
5 for (int i = 1; i <= 5; i++){
6     flowLayoutP.add(new JButton("Button " + i));
7 }
```



Gestionarul de aranjare GridLayout

- Aranjează componentele într-o grilă cu număr fix de rânduri și coloane
- Redimensionează fiecare componentă astfel încât ele să aibă toate aceeași mărime
- Extinde fiecare componentă pentru a umple toată zona alocată lui
- Adăugarea de componente, rând cu rând, de la stânga la dreapta:



```
1 JPanel gridLayoutP = new JPanel(new GridLayout(2, 3, 5, 5));
2 gridLayoutP.setBackground(new Color(100, 80, 210));
3 gridLayoutP.setBorder(BorderFactory.createTitledBorder("GridLayout"));
4 for (int i = 1; i <= 6; i++) {
5     gridLayoutP.add(new JButton("Cell " + i));
6 }
```

Gestionarul de aranjare BorderLayout

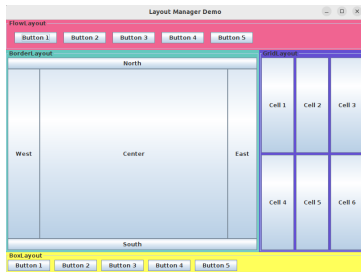
- Gestionarul de aranjare BorderLayout aranjează componentele dintr-un container într-un singur rând sau o singură coloană.
- Spațierea și alinierea pe fiecare rând sau coloană poate fi controlată individual
- Containerele care folosesc BorderLayout pot fi imbricate unul în altul pentru a produce aranjamente complexe
- Constructor:

```
1 public BorderLayout(Container c, int direction);
```

- Parametrul *direction* poate fi *X_AXIS* sau *Y_AXIS*
- Se pot folosi zone rigide (*rigid areas*) și zone "lipicioase" (*glue regions*) pentru a spația componentele într-un BorderLayout

Gestionarul de aranjare BorderLayout

```
1 //... modificati in exemplul C10-gui-4
2 JPanel boxLayoutP = new JPanel();
3 boxLayoutP.setBackground(new Color(255, 255, 90));
4 boxLayoutP.setBorder(BorderFactory.createTitledBorder("BoxLayout"));
5 boxLayoutP.setLayout(new BoxLayout(boxLayoutP, BoxLayout.X_AXIS));
6 for (int i = 1; i <= 5; i++) {
7     boxLayoutP.add(new JButton("Button " + i));
8     //boxLayoutP.add(Box.createVerticalStrut(10));
9     //boxLayoutP.add(Box.createHorizontalGlue());
10    boxLayoutP.add(Box.createRigidArea(new Dimension(10, 0)));
11 }
```

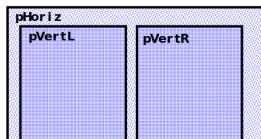


Combinarea gestionărilor de aranjare

- Câteodată e util să creăm mai multe containere unul în altul, fiecare cu propriul gestionar de aranjare
- Spre exemplu, panoul de nivelul cel mai înalt ar putea folosi o aranjare de tipul cutie orizontală, iar în el ar putea fi două sau mai multe panouri cu aranjarea tip cutie verticală
- Rezultatul este controlul complet al spațierii pe ambele dimensiuni

Combinarea gestionărilor de aranjare: C10-gui-5

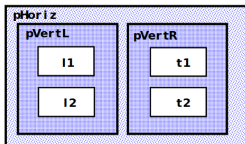
```
1 public static void main(String[] args) {
2     JFrame frame = new JFrame("Containere Si Aranjari Imbricate");
3     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4     frame.setSize(400, 100);
5     // Creaza un nou panou de nivel sus
6     JPanel pHoriz = new JPanel();
7     pHoriz.setLayout(new BoxLayout(pHoriz, BoxLayout.X_AXIS));
8     frame.add( pHoriz );
9     // Creaza doua panouri subordonate
10    JPanel pVertL = new JPanel();
11    JPanel pVertR = new JPanel();
12    pVertL.setLayout(new BoxLayout(pVertL, BoxLayout.Y_AXIS));
13    pVertR.setLayout(new BoxLayout(pVertR, BoxLayout.Y_AXIS));
14    // Adauga la pHoriz cu spatiu oorizontal // intre panouri
15    pHoriz.add( pVertL );
16    pHoriz.add( Box.createRigidArea(new Dimension(20,0)) );
17    pHoriz.add( pVertR );
```



Combinarea gestionărilor de aranjare

Exemplu C10-gui-5

```
1 //... continuare main
2 // Creaza cimpul grade Celsius
3 JLabel l1 = new JLabel("deg C:", JLabel.RIGHT);
4 pVertL.add( l1 );
5 JTextField t1 = new JTextField("0.0",10);
6 pVertR.add( t1 );
7
8 // Creaza cimpul grade Fahrenheit
9 JLabel l2 = new JLabel("deg F:", JLabel.RIGHT);
10 pVertL.add( l2 );
11 JTextField t2 = new JTextField("32.0",10);
12 pVertR.add( t2 );
13 frame.setVisible(true);
14 }
```



Componente GUI

Controale pentru alegeri: Butoane Radio

- Pentru seturi de mici dimensiuni de variante mutual exclusive folosim butoane radio sau o cutie combo
- Într-un set de butoane radio, doar unul poate fi selectat la un moment dat
- Dacă este selectat un alt buton, cel selectat anterior este automat deselectat (turned off)
- Gruparea butoanelor nu pune butoanele apropiate unul de altul pe container
- Trebuie să le aranjăm noi pe ecran
- **isSelected()**: se apelează pentru a afla dacă un anumit buton este curent selectat sau nu

```
1 if (largeButton.isSelected()) size = LARGE_SIZE;
```

- Apelăm **setSelected(true)** pe un buton radio din grup înainte de a face vizibil cadrul care conține butoanele

Controale pentru alegeri: Butoane Radio - Exemplu C10-gui-6

```
1 import javax.swing.*; import java.awt.*;
2 public class Main {
3     public static void main(String[] args) {
4         JFrame frame = new JFrame("Controale Radio, Check si Combo");
5         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         frame.setSize(600, 300);
7         frame.setLayout(new GridLayout(4, 1));
8         JPanel radioPanel = new JPanel();
9         radioPanel.setBorder(BorderFactory.createTitledBorder("PizzaSize"));
10        JRadioButton smallButton = new JRadioButton("Small");
11        JRadioButton mediumButton = new JRadioButton("Medium");
12        JRadioButton largeButton = new JRadioButton("Large");
13        ButtonGroup sizeGroup = new ButtonGroup();
```

```
sizeGroup.add(smallButton);
sizeGroup.add(mediumButton);
sizeGroup.add(largeButton);
radioPanel.add(smallButton);
radioPanel.add(mediumButton);
radioPanel.add(largeButton);
smallButton.setSelected(true);
```



Căsuțe de bifare: JCheckBox

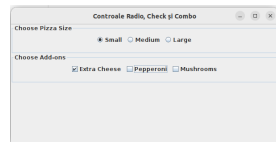
- Au două stări: marcat (checked) și nemarcat
- Pentru o alegere din doua variante posibile folosim o casuta de bifare (checkbox)
- Folosim un grup de casute de bifare atunci când o alegere nu exclude o alta
- Exemplu: "bold" și "italic" la alegerea stilului unui font
- Cum le construim:

```
1 JCheckBox italicCheckBox = new JCheckBox("Italic");
```

Căsuțe de bifare: JCheckBox

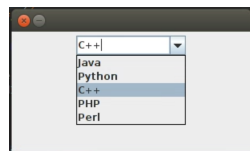
Exemplu C10-gui-6

```
1 //... completati in main
2 JPanel checkBoxPanel = new JPanel();
3 checkBoxPanel.setBorder(BorderFactory.
    createTitledBorder("Choose Add-ons"));
4 JCheckBox cheeseCB=new JCheckBox("Extra Cheese")
    ;
5 JCheckBox pepperoniCB=new JCheckBox("Pepperoni")
    ;
6 JCheckBox mushroomCB=new JCheckBox("Mushrooms");
7 checkBoxPanel.add(cheeseCB);
8 checkBoxPanel.add(pepperoniCB);
9 checkBoxPanel.add(mushroomCB);
10 // ...
11 frame.add(radiusPanel);
12 frame.add(checkBoxPanel);
13 frame.setVisible(true);
```



Căsuțe Combo: JComboBox

- Pentru un număr mare de opțiuni, folosim o casuta combo (combo box)
- "Combo": combinație de listă cu câmp text
- Câmpul text afișează numele selecției curente
- Dacă căsuța combo este editabilă, atunci utilizatorul poate să-și tasteze propria selecție
- Folosim metoda **setEditable()**
- Textele alegerilor le adăugăm folosind metoda **addItem()**:



```
1 JComboBox facenameCombo = new JComboBox();  
2 facenameCombo.addItem("C++");  
3 facenameCombo.addItem("Java");
```

- Obținem alegerea utilizatorului cu **getSelectedItem()** (tipul returnat de aceasta este Object)

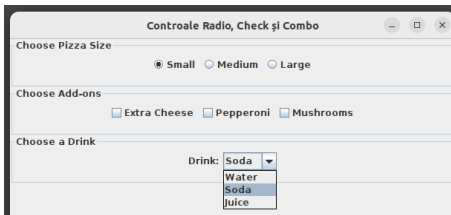
```
1 String selectedString = (String) facenameCombo.getSelectedItem();
```

- Selectăm un element cu **setSelectedItem()**

Căsuțe Combo: JComboBox

Exemplu C10-gui-6

```
1 //... completati in main
2 JPanel comboBoxPanel = new JPanel();
3 comboBoxPanel.setBorder(BorderFactory.createTitledBorder("Choose drink"
    "));
4 String[] drinks = {"Water", "Soda", "Juice"};
5 JComboBox<String> drinkComboBox = new JComboBox<>(drinks);
6 comboBoxPanel.add(new JLabel("Drink:"));
7 comboBoxPanel.add(drinkComboBox);
8 // ...
9 frame.add(radiusPanel);
10 frame.add(checkBoxPanel);
11 frame.add(comboBoxPanel);
12 frame.setVisible(true);
```



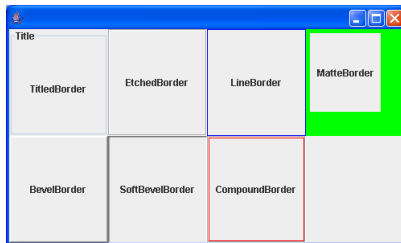
Margini (Borders)

- Punem o margine în jurul panoului pentru a grupa vizual conținutul său
- De exemplu *EtchedBorder*: efect tridimensional de gravare
- Se poate adăuga margine la oricare componentă, dar cel mai adesea se face pentru panouri:

```
1 Jpanel panel = new JPanel ();  
2 panel.setBorder(new EtchedBorder ());
```

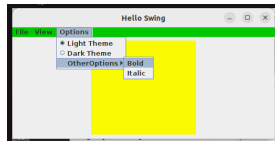
- *TitledBorder*: o margine cu titlu:

```
1 panel.setBorder(new TitledBorder(new EtchedBorder(), "Size"));
```



Meniuri

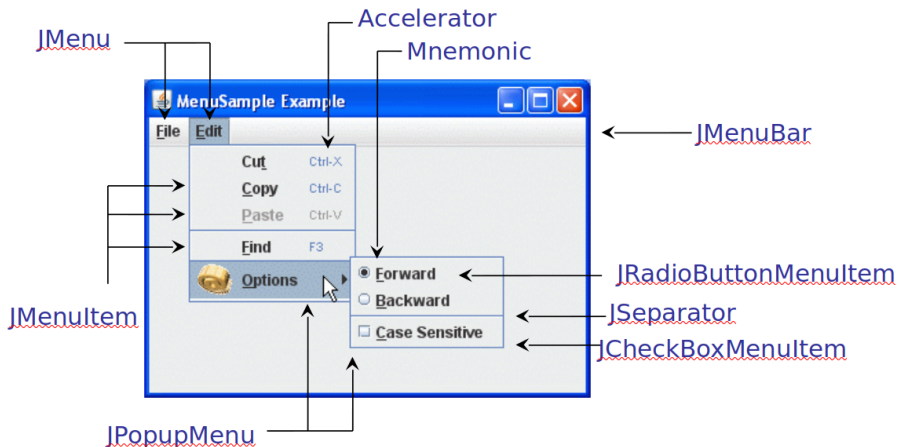
- Fereastra conține o bară de meniu
- Bara de meniu conține meniuri
- Meniul conține submeniuri și elemente de meniu
 - Meniuri Pull-Down
 - JRadioButtonMenuItem



Exemplu cu meniuri: C10-gui-7-meniuri

```
1 JFrame frame = new JFrame("Hello Swing");
2 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
3 frame.setSize(500,250);
4 JMenuBar menuBar = new JMenuBar();
5 menuBar.setOpaque(true);
6 menuBar.setBackground(new Color(0, 200, 0));
7 menuBar.setPreferredSize(new Dimension(200, 20));
8
9 //Creaza meniu pentru File
10 JMenu fileM = new JMenu("File");
11 JMenuItem newItem = new JMenuItem("New");
12 JMenuItem openItem = new JMenuItem("Open");
13 JMenuItem saveItem = new JMenuItem("Save");
14 JMenuItem exitItem = new JMenuItem("Exit");
15 fileM.add(newItem); fileM.add(openItem); fileM.add(saveItem);
16 fileM.addSeparator(); // Add a separator
17 fileM.add(exitItem);
18
19 // Adauga meniul la bara de meniuri
20 menuBar.add(fileM);
21
22 // ... vedeti restul codului pe moodle
```

Meniuri: componente posibile



Zone de text: JTextArea cuprinde mai multe linii de text

- Putem preciza numărul de rânduri și coloane:

```
1 final int ROWS = 10; final int COLUMNS = 30;  
2 JTextArea textArea = new JTextArea(ROWS, COLUMNS);
```

- Numărul de caractere pe linie pentru un obiect JTextField sau JTextArea este numărul de spații em (un spațiu em este spațiul necesar cuprinderii unei litere majuscule M (cea mai lată din alfabet))
- O linie pentru 20 M va fi aproape întotdeauna capabilă să conțină mai mult de 20 caractere
- **setText():** pentru a seta textul unui câmp sau unei zone de text
- **append():** adaugă text la sfârșitul unei zone de text
- Folosim caractere newline pentru a separa liniile:

```
1 textArea.append(account.getBalance() + "\n");
```

- Dacă o folosim doar pentru afișare: *textArea.setEditable(false);*
- Ca să adăugăm scroll bars la o zonă text:

```
1 JTextArea textArea = new JTextArea(ROWS, COLUMNS);  
2 JScrollPane scrollPane = new JScrollPane(textArea);
```

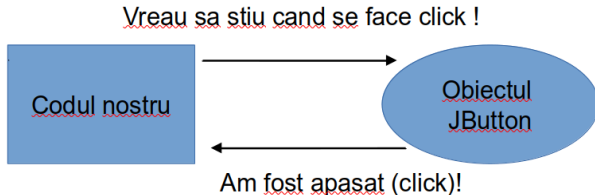
Tratarea Evenimentelor

Tratarea Evenimentelor

- Legătura dintre partea de vizualizare și modelul problemei se face prin transmiterea de evenimente atunci când utilizatorul interacționează cu interfața
- Ex. Click pe un buton, selectarea unui checkbox, apăsarea unei taste etc.
- În Swing există o delimitare clară între interfață și implementare (codul ce trebuie rulat în momentul în care un eveniment se întâmplă sa apară).
- Fiecare componentă Swing poate raporta toate evenimentele ce apar în dreptul ei, si le poate raporta în mod individual, astfel încât să poată fi tratate doar evenimentele de interes.

Tratarea Evenimentelor: C10-gui-8

- Faceti o aplicație GUI simplă care să conțină un singur buton.
- Cand se apasă butonul cu mouse-ul schimbați textul afisat de buton !
- Cum știm când se apasă butonul ?



Ascultători: Listener

- Se apelează un ascultător atunci când utilizatorul interacționează cu interfața ceea ce provoacă un eveniment.
- Deși evenimentele provin de obicei din interfața utilizator, ele pot avea și alte surse (D.e., un contor de timp (Timer)).

- Exemplu de ascultător pentru un buton:

```
1 b.addActionListener(obiectAscultator);
```

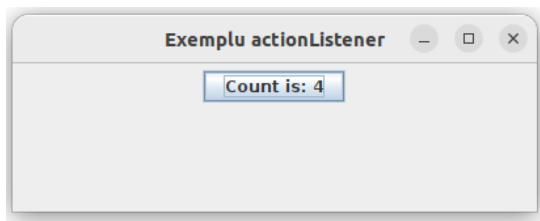
- unde `obiectAscultator` este de tipul unei clase care **implementează interfața `ActionListener`**
- La clic pe buton se face un apel la metoda **`actionPerformed()`** definită în clasa obiectului ascultător. Metodei i se transmite ca parametru un obiect **`ActionEvent`**.

Exemplu ActionListener: C10-gui-8

```
1 public class MainWindow extends JFrame {
2     static int count = 0;
3     JButton b;
4     MainWindow(String title){
5         this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         this.setSize(400,150);
7         this.setTitle(title);
8         b = new JButton("Count = " + count);
9         JPanel contentPane = new JPanel(new BorderLayout());
10        contentPane.add(b);
11        b.addActionListener(new CountListener());
12        this.setContentPane(contentPane);
13    }
14    class CountListener implements ActionListener {
15        public void actionPerformed(ActionEvent e){
16            //fa ceva cand se apasa butonul, ex
17            ++count;
18            b.setText("Count is: " + count );
19        }
20    }
21 }
```

Exemplu ActionListener: C10-gui-8

```
1 public class Main {  
2     public static void main(String[] args) {  
3         MainWindow mw = new MainWindow("Exemplu ActionListener");  
4         mw.setVisible(true);  
5     }  
6 }
```

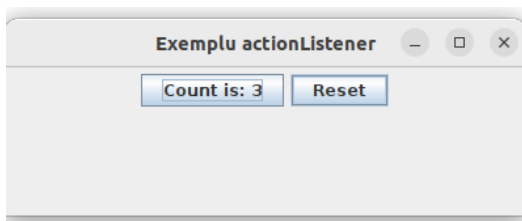


Adăugați un buton de reset pentru counter!

```
1 public class MainWindow extends JFrame {
2     static int count = 0;
3     JButton b;  JButton r;
4     MainWindow(String title){
5         this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6         this.setSize(400,150);
7         this.setTitle(title);
8         b = new JButton("Count = " + count);
9         r = new JButton("Reset");
10        JPanel contentPane = new JPanel(new BorderLayout());
11        contentPane.add(b);
12        contentPane.add(r);
13        b.addActionListener(new CountListener());
14        r.addActionListener(new ResetListener());
15        this.setContentPane(contentPane);
16    }
17    class CountListener implements ActionListener { // ... }
18    class ResetListener implements ActionListener {
19        public void actionPerformed(ActionEvent e){
20            //fa ceva cand se apasa butonul, ex
21            count = 0;
22            b.setText("Count is: " + count );
23        }
24    }
```

Exemplu ActionListener: C10-gui-8

```
1 public class Main {  
2     public static void main(String[] args) {  
3         MainWindow mw = new MainWindow("Exemplu ActionListener");  
4         mw.setVisible(true);  
5     }  
6 }
```



Același exemplu dar cu clase anonime pentru ActionListener: C10-gui-9

```
1 b = new JButton("Count = " + count);
2 r = new JButton("Reset");
3 JPanel contentPane = new JPanel(new FlowLayout());
4 contentPane.add(b);
5 contentPane.add(r);
6 b.addActionListener(new ActionListener() {
7     @Override
8     public void actionPerformed(ActionEvent actionEvent) {
9         ++count;
10        b.setText("Count is: " + count );
11    }
12 });
13 r.addActionListener(new ActionListener() {
14     @Override
15     public void actionPerformed(ActionEvent actionEvent) {
16         count = 0;
17         b.setText("Count is: " + count );
18     }
19 });
20 this.setContentPane(contentPane);
```

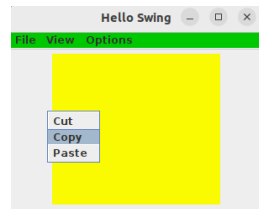
Adăugare listener pe alte tipuri de componente

Reluați exemplul C10-gui-7

La click-dreapta pe panel se deschide un meniu !

```
1 JPanel contentPane = new JPanel();  
2 // ...  
3 // Create a JPopupMenu  
4 JPopupMenu popupMenu = new JPopupMenu();  
5 JMenuItem cutItem = new JMenuItem("Cut");  
6 JMenuItem copyItem = new JMenuItem("Copy");  
7 JMenuItem pasteItem = new JMenuItem("Paste");  
8 popupMenu.add(cutItem);  
9 popupMenu.add(copyItem);  
10 popupMenu.add(pasteItem);
```

```
1 // Add a listener to show the popup menu  
2 contentPane.setComponentPopupMenu(popupMenu);  
3  
4 contentPane.addMouseListener(new java.awt.event.MouseAdapter() {  
5     public void mouseReleased(java.awt.event.MouseEvent e) {  
6         if (e.isPopupTrigger()) {  
7             popupMenu.show(e.getComponent(), e.getX(), e.getY());  
8         }  
9     }  
10 }
```



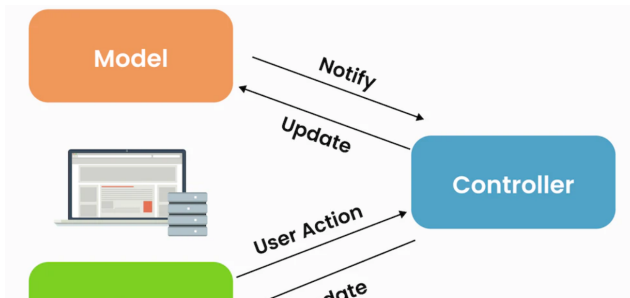
Tipuri de evenimente

<i>Control utilizator</i>	<i>addXXXListener</i>	<i>Metoda în ascultător (listener)</i>
JButton JTextField JMenuItem	addActionListener()	actionPerformed(ActionEvent e)
JSlider	addChangeListener()	stateChanged(ChangeEvent e)
JCheckBox	addItemListener()	itemStateChanged()
key on component	addKeyListener()	keyPressed(), keyReleased(), keyTyped()
mouse on component	addMouseListener()	mouseClicked() , mouseEntered(), mouseExited(), mousePressed(), mouseReleased()
mouse on component	addMouseMotionListener()	mouseMoved(), mouseDragged()
JFrame	addWindowListener()	windowClosing(WindowEvent e), ...

Aplicație MVC

Swing și arhitectura MVC (Model-Vizualizare-Controller)

- Arhitectura Swing își are rădăcinile în arhitectura model-view-controller (MVC) care a fost introdus inițial în limbajul SmallTalk.
- Arhitectura MVC cere ca o aplicație vizuală să fie divizată în trei părți separate:
 - Un model care reprezintă intern datele aplicației
 - O vizualizare (view) – reprezentarea vizuală a datelor respective.
 - Un controlor (controller) care preia intrarea de la utilizator și o transpune în schimbări în model.



Modelul

- Majoritatea programelor trebuie să facă ceva util, nu să fie "o altă față frumoasă"
 - dar există câteva excepții
 - au existat programe utile cu mult înaintea apariției GUI
- Modelul este partea care face treaba - adică modelează problema care este în curs de soluționare prin program
- Modelul ar trebui să fie independent atât de Controlor cât și de Vizualizare dar poate să le furnizeze amândurora servicii (metode)
- Independența furnizează flexibilitate și robustețe

Controlorul

- Controlorul decide ce urmează să facă modelul
- Adesea, utilizatorul are controlul prin intermediul unei GUI în acest caz, GUI și Controlorul sunt adesea același
- Controlorul și Modelul pot fi separate aproape întotdeauna (ce trebuie făcut în raport cu în ce fel trebuie făcut)
- Proiectarea Controlorului depinde de model
- Modelul nu ar trebui să depindă de Controlor

Vizualizarea

- Tipic, utilizatorul trebuie să poată vedea, sau vizualiza, ce face programul
- Vizualizarea arată ce face Modelul
- Vizualizarea este un observator pasiv; ea nu ar trebui să afecteze modelul
- Modelul trebuie să fie independent de vizualizare (dar îi poate furniza metode de acces)
- Vizualizarea nu trebuie să afișeze ce crede Controlorul că se întâmplă

Combinarea Controlorului și a Vizualizării

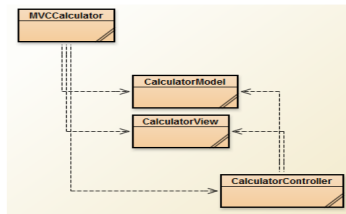
- Uneori Controlorul și Vizualizarea sunt combinate, mai ales în programe de mici dimensiuni
- Combinarea Controlorului și a Vizualizării este potrivită dacă cele două sunt foarte interdependente
- Modelul trebuie să rămână independent
- NU amestecați niciodată codul din Model cu codul GUI!

Separarea preocupărilor

- Ca întotdeauna, dorim independența codului
- Modelul nu trebuie contaminat cu cod din control sau din vizualizare
- Vizualizarea trebuie să reprezinte Modelul așa cum este în realitate, nu vreo stare pe care și-o amintește
- Controlorul trebuie să converseze cu Modelul și Vizualizarea, nu să le manipuleze
- Controlorul poate seta variabile pe care Modelul și Vizualizarea le pot citi

Exemplu de problemă MVC

Implementarea unui calculator simplificat.



Vedeți codul din exemplul C10-gui-10-mvc.

Exemplu MVC: Modelul

```
1 public class CalculatorModel {
2     private int calculationValue;
3     public void addTwoNumbers(int firstNumber, int secondNumber)
4     {
5         calculationValue = firstNumber + secondNumber;
6     }
7
8     public int getCalculationValue()
9     {
10         return calculationValue;
11     }
12 }
```

Exemplu MVC: Vizualizarea

```
1 import java.awt.event.ActionListener;
2 import javax.swing.*;
3 public class CalculatorView extends JFrame{
4     private JTextField firstNumber = new JTextField(10);
5     private JLabel additionLabel = new JLabel("+");
6     private JTextField secondNumber = new JTextField(10);
7     private JButton calculateButton = new JButton("Calculate");
8     private JTextField calcSolution = new JTextField(10);
9
10    CalculatorView(){
11        JPanel calcPanel = new JPanel();
12        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13        this.setSize(600, 200);
14
15        calcPanel.add(firstNumber);
16        calcPanel.add(additionLabel);
17        calcPanel.add(secondNumber);
18        calcPanel.add(calculateButton);
19        calcPanel.add(calcSolution);
20        this.add(calcPanel);
21    }
```

Exemplu MVC: Vizualizarea

```
1 public int getFirstNumber() {
2     return Integer.parseInt(firstNumber.getText());
3 }
4 public int getSecondNumber() {
5     return Integer.parseInt(secondNumber.getText());
6 }
7 public int getCalcSolution() {
8     return Integer.parseInt(calcSolution.getText());
9 }
10 public void setCalcSolution(int solution) { calcSolution.setText(
    Integer.toString(solution));
11 }
12 void addCalculateListener(ActionListener listenForCalcButton) {
13     calculateButton.addActionListener(listenForCalcButton);
14 }
15 void displayErrorMessage(String errorMessage) {
16     JOptionPane.showMessageDialog(this, errorMessage);
17 }
18 }
```

Exemplu MVC: Controlorul

```
1 import java.awt.event.ActionEvent;
2 import java.awt.event.ActionListener;
3 public class CalculatorController {
4     private CalculatorView theView;
5     private CalculatorModel theModel;
6     public CalculatorController(CalculatorView theView, CalculatorModel
       theModel) {
7         this.theView = theView;
8         this.theModel = theModel;
9         this.theView.addCalculateListener(new CalculateListener());
10    }
11    class CalculateListener implements ActionListener{
12        public void actionPerformed(ActionEvent e) {
13            int firstNumber, secondNumber = 0;
14            try{
15                firstNumber = theView.getFirstNumber();
16                secondNumber = theView.getSecondNumber();
17                theModel.addTwoNumbers(firstNumber, secondNumber);
18                theView.setCalcSolution(theModel.getCalculationValue());
19            }
20            catch (NumberFormatException ex){
21                System.out.println(ex);
22                theView.displayErrorMessage("You Need to Enter 2 Integers");
23            }
24        }
25    }
```

Exemplu MVC: Clasa principală

```
1 public class Main {
2     public static void main(String[] args) {
3
4         CalculatorView theView = new CalculatorView();
5
6         CalculatorModel theModel = new CalculatorModel();
7
8         CalculatorController theController = new CalculatorController(
9             theView, theModel);
10
11         theView.setVisible(true);
12     }
13 }
```